

SHIELD Phase 4 — PTQ 양자화 & 경량 모델 배포 시 행착오 정리

작성일: 2026-03-20 | 범위: Phase 4.1 ONNX 변환 → PTQ 평가 → SpecM-v3 재학습 → RPi5 추론 스크립트

최종 결과: RPi5 2-model ICWMV (MNV2-Dynamic + SpecM-v3-Dynamic) = macro-F1 — 서버 4-model 대비 Δ -0.29%p

SHIELD Phase 4 — PTQ 양자화 & 경량 모델 배포 시행착오 정리

작성일: 2026-03-20

범위: Phase 4.1 ONNX 변환 → Phase 4.2 PTQ 평가 → SpecM-v3 재학습 → RPi5 추론 스크립트

결론: RPi5 2-model ICWMV (MNV2-Dynamic + SpecM-v3-Dynamic) = **96.19%** macro-F1, 서버 4-model 대비 **Δ -0.29%p**

1. 배경 — 왜 경량화가 필요한가

SHIELD 연구의 핵심 목표는 이미지 위조 탐지를 **서버 없이 Raspberry Pi 5에서 실행**하는 것이다.

서버 기준 4-model ICWMV(MNV2 + CLIP + SpecM + SpecG)는 96.48% macro-F1이지만 총 모델 크기 335MB, 이미지당 ~790ms로 RPi5 실시간 처리가 불가능하다.

RPi5 제약 조건

항목	제약
메모리	8GB RAM, 스왑 없음
CPU	Cortex-A76 × 4, 2.4GHz
목표 latency	이미지 1장 ≤ 200ms
목표 정확도	≥ 95% macro-F1 (서버 대비 -1.5%p 이내)

2. Phase 4.1 — ONNX 변환 및 CPU 벤치마크

2.1 작업 내용

4개 모델(MNV2, SpecM, SpecG, CLIP)을 모두 ONNX opset 14로 변환 후 1-thread CPU 추론 속도를 측정했다.

2.2 결과

모델	크기	서버 (1-thread)	RPi5 추정 (×4)	적합 여부
MNV2 dual-stream	22.5 MB	14 ms	~56 ms	☑
SpecM (RGB+SRM+DCT)	30 MB	21 ms	~84 ms	☑
SpecG (FastViT+MobileCLIP)	141.5 MB	200 ms	~800 ms	☑
CLIP (MobileCLIP-S2 ft4)	141.3 MB	198 ms	~792 ms	☑

결론: SpecG와 CLIP은 RPi5에서 이미지당 ~800ms로 예산 초과. **MNV2 + SpecM 조합만 RPi5 적합**. 합산 ~140ms.

3. Phase 4.2 — PTQ INT8 양자화 1차 시도 (실패 → 원인 발견)

3.1 첫 번째 평가 결과 (2026-03-19, `quant\_accuracy\_20260319\_145436.json`)

Dynamic INT8 변환 후 정확도를 평가했더니 MNV2가 예상과 전혀 다른 수치가 나왔다.

모델	FP32	Dynamic INT8	Δ
MNV2	76.11%	76.13%	+0.02%p
SpecM	65.54%	65.54%	0.00%p

MNV2 FP32가 76.11%인데, backbone_eval에서 PyTorch 기준 **95.82%**였다. 약 **20%p** 차이.

3.2 원인 분석 — 이중 정규화(Double Normalization) 버그

`run_quant_accuracy_eval.py`의 `load_image()` 함수가 `imagenet` 정규화를 적용한 뒤 ONNX 추론에 넘겼는데, ONNX 모델 내부에도 동일한 정규화가 포함되어 있었다.

```
# 문제 코드 — 외부에서 정규화 적용
x = (x - IMAGENET_MEAN) / IMAGENET_STD # ← 문제

# ONNX 모델 내부 (run_onnx_export.py)
```

```
def forward(self, x):
    rgb = normalize_batch(x, IMAGENET_MEAN, IMAGENET_STD) # ← 모델 내부에도 동일 정규화
```

입력이 두 번 정규화되어 픽셀 값이 완전히 틀어졌다.

확인 방법: backbone_eval JSONL의 PyTorch 예측값으로 macro-F1을 직접 계산했더니 95.82%가 나와서 ONNX 이전 문제임을 확인했다.

3.3 수정

`load_image()` 함수에서 `imagenet` 정규화를 제거하고, MNV2와 SpecM에 대해 `norm="raw"` (0~1 원본값) 적용으로 수정했다.

4. Phase 4.2 — PTQ INT8 양자화 2차 평가 (이중 정규화 수정 후)

4.1 수정 후 결과 (2026-03-19, ``quant_accuracy_20260319_163814.json``)

모델	FP32	Dynamic INT8	Δ	Static INT8	Δ
MNV2	95.37%	95.37%	+0.00%p	81.95%	-13.42%p
SpecM	65.11%	65.14%	+0.03%p	64.26%	-0.85%p
SpecG	87.67%	87.84%	+0.17%p	40.44%	-47.23%p
CLIP	95.32%	95.38%	+0.06%p	32.07%	-63.25%p

4.2 핵심 발견

① Dynamic INT8 = 무손실

4개 모델 모두 FP32 대비 $\Delta \leq +0.17\%$ p. 정확도 손실 없이 양자화 성공.

② Static INT8 — FastViT 계열 완전 붕괴

SpecG(FastViT+MobileCLIP)와 CLIP의 Static INT8은 각각 -47%p, -63%p로 완전 붕괴. 원인:

- FastViT의 attention layer는 activation 범위가 넓고 분포가 비정규적
- Static 양자화의 캘리브레이션 데이터(300장)로는 범위 추정 불가
- ConvInteger 미구현 오류(`/srm_extractor/Conv_quant`)도 발생 →
`op_types_to_quantize=["Gemm", "MatMul"]` 로 Conv 제외 필요

③ SpecM openSDI 약점

SpecM FP32에서도 opensdi macro-F1 = 41%, aigenproxy = 53%. 이는 양자화 문제가 아닌 **모델 자체의 일반화 한계**:

- SpecM 학습 데이터: CASIA2(copy-move/splicing) + IMD2020(generative inpainting)
- openSDI 조작 방식: Stable Diffusion 1.5 기반 inpainting → 완전히 다른 도메인

5. ICWMV 2-model 조합 성능 평가

5.1 조합 방식

SpecM JSONL에 ai_generated 이미지가 없음(binary: auth/manip만 평가). 따라서:

- **auth/manip 이미지**: MNV2 + SpecM class-wise weighted average
- **ai_generated 이미지**: MNV2 단독 (SpecM 기여 없음)

ICWMV 융합 수식:

```

s_auth = (MNV2[auth] + w × SpecM[auth]) / (1 + w)
s_manip = (MNV2[manip] + w × SpecM[manip]) / (1 + w)
s_aigen = MNV2[aigen]
→ renormalize → argmax

```

5.2 MNV2 + SpecM-v2 결과

DS	MNV2 단독	ICWMV v2	Δ
base	94.38%	95.26%	+0.88%p
dsC	97.89%	98.11%	+0.22%p
opensdi	94.89%	93.32%	-1.56%p
aigenproxy	96.10%	95.88%	-0.23%p
avg	95.81%	95.64%	-0.17%p

openSDI에서 SpecM이 악영향을 미쳐 MNV2 단독보다 낮아졌다.

서버 4-model(96.48%) 대비 **Δ-0.84%p**.

6. SpecM-v3 재학습 — Authentic OOD 강건화

6.1 문제 진단

SpecM openSDI 41% macro-F1의 근본 원인을 파악했다.

- backbone_eval 재계산으로 PyTorch SpecM-v2 openSDI auth_recall = **11%**
- 즉, openSDI 실제 authentic 이미지의 89%를 manipulated로 오분류

- 학습 authentic 데이터가 CASIA2 실내/사물 중심 → openSDI의 자연/야외 authentic과 domain gap

6.2 데이터 분석

데이터셋	장수	비고
OpenSDID_subset	900장 (300×3)	전부 eval set — 훈련 불가 (중복 100%)
GenImage_subset/BigGAN/val/nature	6,000장	ImageNet val 실제 자연 사진 — 훈련 가능

OpenSDI 전체 subset이 eval set이므로 직접 추가 불가. **GenImage nature 사진으로 authentic 다양화** 전략 선택.

6.3 SpecM-v3 변경 사항

v2 대비 변경점 (`experiments/train_specialist_m_v3.py`):

항목	v2	v3
Authentic 학습 데이터	CASIA2 Au (7,491장)	CASIA2 Au (7,491장) + GenImage nature (3,000장)
Augmentation 추가	—	RandomErasing(p=0.3, scale 2~20%)
총 학습 데이터	~12,614장	~17,324장
Authentic 도메인	실내/사물 편향	실내+자연/야외 혼합

RandomErasing 추가 이유: 조작 이미지에서 패치 단위 inpainting을 학습하기 위해 authentic 이미지에 임의 영역을 지워 모델이 지역 패턴을 더 학습하도록 유도.

6.4 v3 학습 결과

```
best manip_f1 = 0.7832 (epoch 11 / 30)
```

DS	auth_recall	manip_recall	manip_f1
base	73.4%	95.2%	85.8%
dsC	75.0%	92.7%	85.1%
opensdi	62.0%	69.7%	67.1%
aigenproxy	93.7%	97.7%	95.8%

openSDI auth_recall: **11% → 62% (+51%p)** — 핵심 목표 달성.

7. SpecM-v3 ONNX 변환 및 시행착오

7.1 1차 시도 실패 — New torch.onnx export API

```
torch.onnx.export(..., opset_version=17, ...)
# → shape inference error: (3840) vs (256)
```

PyTorch 2.x의 새 torch.export 기반 exporter는 onnxruntime quantization과 호환 문제가 있다.

7.2 2차 시도 실패 — quantize_dynamic 인자 오류

```
quantize_dynamic(..., optimize_model=True)
# → TypeError: unexpected keyword argument 'optimize_model'
```

현재 설치된 onnxruntime 버전에 해당 파라미터 없음.

7.3 3차 시도 실패 — ConvInteger 미구현

```
onnxruntime.capi.onnxruntime_pybind11_state.NotImplemented:
Could not find an implementation for ConvInteger(10) node '/srm_extractor/Conv_quant'
```

SRM Extractor의 Conv2d를 INT8으로 변환하면 ConvInteger op를 사용하는데 해당 ORT 빌드에서 미구현 상태.

7.4 최종 해결 — `dynamo=False` + `op\_types\_to\_quantize=["Gemm","MatMul"]`

기존 `run_quantization.py`의 방식 참고:

```
# ONNX export: TorchScript 기반 opset 14
torch.onnx.export(..., opset_version=14, dynamo=False, ...)

# 양자화: Gemm/MatMul만 (Conv 제외)
quantize_dynamic(
    op_types_to_quantize=["Gemm", "MatMul"],
    weight_type=QuantType.QInt8,
)
```

FP32 vs Dynamic INT8 일치율 100%(100장) 확인.

8. 최종 성능 비교 — v2 vs v3

8.1 ICWMV 2-model 결과 (MNV2 + SpecM-v3 Dynamic)

DS	MNV2 단독	v2 ICWMV	v3 ICWMV	$\Delta(v3-v2)$
base	94.38%	95.26%	94.65%	-0.61%p
dsC	97.89%	98.11%	98.22%	+0.11%p
opensdi	94.89%	93.32%	94.68%	+1.35%p
aigenproxy	96.10%	95.88%	97.22%	+1.34%p
avg	95.81%	95.64%	96.19%	+0.55%p

8.2 전체 시스템 비교

시스템	모델 수	avg macro-F1	서버 대비
서버 4-model ICWMV	4	96.48%	기준
RPi5 MNV2 + SpecM-v2 Dynamic	2	95.64%	-0.84%p
RPi5 MNV2 + SpecM-v3 Dynamic	2	96.19%	-0.29%p
MNV2 단독	1	95.81%	-0.67%p

CLIP + SpecG를 제거하고 2-model로 경량화하면서 서버 수준의 96%를 유지하는 것에 성공했다.

9. RPi5 추론 스크립트 (`inference\_rpi5.py`)

9.1 설계

단일 Python 파일, MAIFS 코드베이스 의존성 없음. RPi5에서 `pip install onnxruntime pillow numpy` 만으로 실행 가능.

의존성:

```
onnxruntime    # ONNX 추론
Pillow         # 이미지 로드
numpy          # 수치 연산
```

9.2 주요 흐름

```
이미지 로드 → [0,1] float32 NCHW
↓
MNV2 ONNX 추론 → softmax → [auth, manip, aigen] probs
SpecM ONNX 추론 → softmax → [auth, manip] probs
↓
ICWMV 융합
  auth/manip: class-wise weighted avg (w=1.0)
  aigen:      MNV2 단독
↓
renormalize → argmax → verdict
```

9.3 사용법

```
# 텍스트 출력
python inference_rpi5.py image.jpg
```

```
# JSON 출력
python inference_rpi5.py image.jpg --json

# RPi5 최적 (4-thread)
python inference_rpi5.py image.jpg --threads 4

# 출력 예시
판정: AUTHENTIC (58.7% 신뢰도)
    authentic=0.587 manipulated=0.350 ai_generated=0.063
추론: 40ms | 모델로드: 193ms
```

10. 남은 과제 및 한계

10.1 확인된 한계

항목	내용
SpecM openSDI manip_recall	69.7% — SD 기반 inpainting 탐지 여전히 어려움
SpecM-v3 base 성능	94.65% (v2 95.26% 대비 -0.61%p) — GenImage 추가로 base 약간 저하
Dynamic INT8 속도	서버에서 v3 FP32(27ms) vs Dynamic(41ms) — ORT overhead. 실제 RPi5에서 재측정 필요
Static INT8	SpecG/CLIP FastViT 계열 붕괴 → QAT 없이는 Static 불가

10.2 다음 단계 (Phase 4.4)

1. **RPi5 실측** — `inference_rpi5.py` + onnxruntime ARM 빌드로 latency/memory 실측
2. **SpecM 추가 학습** — openSDI manip 탐지 개선을 위해 SD inpainting 스타일 데이터 확보
3. **논문 실험 섹션** — PTQ 결과 및 경량화 비용 정량화 표 작성

부록 A — 주요 파일 경로

파일	설명
<code>weights/onnx/mnv2.onnx</code>	MNV2 FP32 ONNX (22.5MB)
<code>weights/onnx_quant/mnv2_int8_dynamic.onnx</code>	MNV2 Dynamic INT8 (6.5MB)
<code>weights/onnx/specm_v3.onnx</code>	SpecM-v3 FP32 ONNX (30.6MB)
<code>weights/onnx_quant/specm_v3_int8_dynamic.onnx</code>	SpecM-v3 Dynamic INT8 (27.7MB)
<code>weights/specialist_m_v3/specialist_m_v3_best.pth</code>	SpecM-v3 PyTorch 체크포인트
<code>experiments/train_specialist_m_v3.py</code>	SpecM-v3 학습 스크립트
<code>experiments/run_quant_accuracy_eval.py</code>	PTQ 정확도 평가 스크립트 (tqdm 추가)
<code>inference_rpi5.py</code>	RPi5 단독 추론 스크립트
<code>experiments/results/onnx_benchmark/quant_accuracy_20260319_163814.json</code>	PTQ 최종 결과

부록 B — 핵심 수치 요약

서버 4-model ICWMV	: 96.48%	(MNV2 + CLIP + SpecM + SpecG)
RPi5 2-model ICWMV (SpecM-v2)	: 95.64%	Δ -0.84%p
RPi5 2-model ICWMV (SpecM-v3)	: 96.19%	Δ -0.29%p ← 최종 배포

MNV2 단독 FP32 : 95.81%

SpecM openSDI auth_recall 개선

v1: ~7% → v2: 11% → v3: 62% (+55%p 총 개선)

RPi5 예상 latency

MNV2-Dynamic : ~56ms

SpecM-v3-Dynamic : ~108ms

합계 : ~164ms (200ms 예산 이내)